

Exercise 1: Centralized State with NgRx SignalStore

Context: Local signals created via `signal()` function as isolated reactive data containers. If multiple components declare separate signals or execute independent `HttpClient` operations for the same state data, they preserve separate in-memory copies. Mutating one local copy leaves the other unmutated. To prevent state drift across shared elements, you must implement a centralized **Singleton Store**—one instance in memory, shared by every component that injects it.

Step 1: Install NgRx SignalStore

Open a terminal in your Angular project folder and install the core NgRx signals package dependency:

```
npm install @ngrx/signals
```

[!NOTE]

Step 2: Build the Centralized Enrollment Store

Create `src/app/store/enrollment.store.ts` to implement a structured, performance-optimized shared state engine:

```
import { computed, inject } from '@angular/core';
import {
  signalStore,
  withComputed,
  withMethods,
  patchState,
  withState,
} from '@ngrx/signals';
import {
  withEntities,
  setAllEntities,
  updateEntity,
} from '@ngrx/signals/entities';
import { rxMethod } from '@ngrx/signals/rxjs-interop';
import { pipe, concatMap, tap, catchError, EMPTY } from 'rxjs';
import { EnrollmentService } from '../services/enrollment.service';
import { Enrollment } from '../models/enrollment.model';

export const EnrollmentStore = signalStore(
  { providedIn: 'root' },

  // withState adds simple properties alongside the entity
  // collection
  withState({ isLoading: false, error: null as string | null }),

  // withEntities creates an O(1) ID-indexed dictionary for the
  // enrollment collection.
```

```

// Internally, it stores { ids: string[], entityMap:
Record<string, Enrollment> }
// so lookups and updates by ID are instant – no array scanning.
withEntities<Enrollment>(),

// withComputed creates read-only derived signals that update
automatically.
// pendingCount recalculates every time the entity collection
changes.
withComputed((store) => ({
  pendingCount: computed(
    () => store.entities().filter(e => e.status ===
'Pending').length
  ),
})),

// withMethods encapsulates asynchronous side effects and
transactional behaviors
withMethods((store, api = inject(EnrollmentService)) => ({
  // Loading Data
  // Why concatMap here? Because concatMap processes one emission
at a time
  // in strict order. If something triggers loadEnrollments()
twice quickly,
  // concatMap waits for the first HTTP response before starting
the second.
  // switchMap would cancel the first request (data loss risk).
  // mergeMap would run both in parallel (race condition risk).
  loadEnrollments: rxMethod<void>(
    pipe(
      tap(() => patchState(store, { isLoading: true, error: null
})),
      concatMap(() =>
        api.getAll().pipe(
          tap(rows => patchState(store, setAllEntities(rows), {
isLoading: false })),
          catchError(err => {
            patchState(store, { isLoading: false, error:
err.message });
            return EMPTY; // EMPTY completes silently so the
rxMethod pipeline survives
          })
        )
      )
    ),
  ),

  // Optimistic Approve
  // Step 1: Instantly flip the status to "Approved" in the
store.
  // Every component reading from the store sees the change
immediately.
  // Step 2: Send the approval to the server.

```

```

// Step 3: If the server rejects it, roll back the status to
"Pending."
approveEnrollment: rxMethod<string>(
  pipe(
    tap(id => {
      // Optimistic update – the UI reacts before the network
      round-trip completes
      patchState(store, updateEntity({ id, changes: { status:
'Approved' } }));
    }),
    concatMap(id =>
      api.approve(id).pipe(
        catchError(err => {
          // Server said no – restore the previous state
          patchState(store, updateEntity({ id, changes: {
status: 'Pending' } }));
          patchState(store, { error: 'Server rejected the
approval. Check enrollment constraints.' });
          return EMPTY;
        })
      )
    )
  )
),
}))
);

```

[!NOTE]

Step 3: Wire the Store into Your Component

Generate the enrollment list component inside your terminal:

```
ng generate component features/enrollment-list
```

Open `src/app/features/enrollment-list/enrollment-list.component.ts` and connect it to your centralized singleton store:

```

import { Component, inject, OnInit } from '@angular/core';
import { EnrollmentStore } from '../../store/enrollment.store';

@Component({
  selector: 'tms-enrollment-list',
  standalone: true,
  imports: [],
  templateUrl: './enrollment-list.component.html'
})
export class EnrollmentListComponent implements OnInit {
  store = inject(EnrollmentStore);

```

```

ngOnInit() {
  this.store.loadEnrollments();
}

onApprove(id: string) {
  this.store.approveEnrollment(id);
}
}

```

Open `src/app/features/enrollment-list/enrollment-list.component.html` and bind the view layout directly to the store's reactive selectors:

```

@if (store.isLoading()) {
  <p>Loading enrollments...</p>
}

@for (enrollment of store.entities(); track enrollment.id) {
  <div class="enrollment-card">
    <span>{{ enrollment.studentName }} – {{ enrollment.courseName }}</span>
    <span class="status">{{ enrollment.status }}</span>

    @if (enrollment.status === 'Pending') {
      <button (click)="onApprove(enrollment.id)">Approve</button>
    }
  </div>
}

@if (store.error()) {
  <p class="error">{{ store.error() }}</p>
}

```

Architectural Architecture & Verification Review

To see the centralized state engine in action, render two separate components on screen that both read data directly from `EnrollmentStore` (e.g., the primary enrollment index list and a dashboard summary counter widget displaying `store.pendingCount()`).

- **Real-Time Synchronization:** Clicking the **Approve** button on an item executes a local modification statement inside your layout views. Without navigating away from the page, reloading your browser, or executing manual refresh triggers, the dashboard counter widget's pending total drops by one automatically.
- **Singleton State Management:** This synchronization works seamlessly because both individual presentation elements inject the exact same singleton store memory instance. The instant `patchState` fires, the underlying entity dictionary updates, causing every component bound to `store.entities()` or `store.pendingCount()` to re-render automatically.

- **Zero Resource Overhead:** This reactive loop updates your application interface with zero manual refresh code, zero duplicated API network calls, and completely eliminates local data state drift.

Note: Cross-tab synchronization (updating two separate browser windows running on different devices simultaneously) requires an active real-time push channel server. You will implement that messaging channel using SignalR. This pattern forms the architectural foundation for everything that follows: performance optimization metrics, enterprise grids, defensive RxJS streams, and real-time sync.